



PracticeDouble

🕒 Created time	July 31, 2026 5:59 AM
📁 Category	CommonVariables
👤 Last edited by	R.A.R.A.
🕒 Last updated time	July 31, 2026 8:57 AM

תרגול מעשי: מערכת חישוב נתוני חיישנים IoT Sensor Metrics

בנה תוכנית Console ב-C# המבצעת את השלבים הבאים:

1. הגדרת נתוני בסיס:

צור שלושה משתנים מטיפוס שלם (`int`) המייצגים זמני תגובה של שרת באלפיות שנייה (ms).

תן להם את הערכים: `43`, `21`, ו-`62`.

2. חישוב ממוצע מדויק:

צור משתנה בשם `averageResponseTime` מטיפוס `double`.

חשב את הממוצע של שלושת משתני ה-`int` שיצרת בשלב 1, והצב את התוצאה בתוך `averageResponseTime`.

דרישת חובה: עליך לוודא שהתוצאה של החישוב כוללת את השארית Decimal point ולא מבוצעת Integer Division שתאבד לנו נתונים.

ישם את הטריק שלמדנו בקוד שהדבקת כדי לאותת לקומפילר שאתה רוצה תוצאה עשרונית.

3. הדפסה:

הדפס לקונסולה את הממוצע שחישבת.

4. המרה מרומזת (Implicit Conversion):

הוכח שניתן להכניס מספר שלם לתוך משתנה `double` ללא בעיה כי אנחנו עוברים מטיפוס פחות ספציפי ליותר ספציפי.

צור משתנה `int` בשם `basePing` עם הערך `100`.

מיד לאחר מכן, צור משתנה `double` בשם `exactBasePing` והצב לתוכו ישירות את המשתנה `basePing`.

5. שימוש בספריית Math:

השתמש בספריית `Math` של .NET כדי לבצע פעולה מתמטית על ה-`averageResponseTime` למשל, תוציא לו שורש ריבועי באמצעות `Math.Sqrt` או שתעגל אותו.

הוכח שהספרייה הזו עובדת באופן טבעי עם טיפוס `double`.

6. שאלת הבנה (כתוב את התשובה כהערה // בתוך הקוד שלך):

נניח והמערכת שלך עכשיו צריכה לחשב את העלות הכספית של תחזוקת השרתים הללו.

האם תשתמש בטיפוס `double` כדי לאחסן את סכומי הכסף?

הסבר בקצרה למה כן או למה לא.

```
1//
:Setting up the base data//
Create three integer variables that represent server response times in millisecond//
.s (ms)
.`Give them the values: `43`, `21`, and `62`//

;int serverResponseTimesInMillisecondsOne = 43

;int serverResponseTimesInMillisecondsTwo = 21
```

```
;int serverResponseTimesInMillisecondsThree = 62
```

```
//2
//Calculating the exact average:
//Create a variable named `averageResponseTime` of type `double`.
//Calculate the average of the three `int` variables you created in step 1, and place the result inside `averageResponseTime`.
//Required:
//You must make sure that the result of the calculation includes the decimal point and that no "integer division" is performed that will lose us data.
//Apply the trick we learned in the code you pasted to signal to the compiler that you want a decimal result.

//Hard coded:
//double averageResponseTime = (43.0 + 21 + 62) / 3;

double averageResponseTime = (serverResponseTimesInMillisecondsOne + serverResponseTimesInMillisecondsTwo + serverResponseTimesInMillisecondsThree) / 3.0;
```

```
//3
//Printing:
//Print the average you calculated to the console.

Console.WriteLine(averageResponseTime);
```

```
//4
//Implicit Conversion:
//Prove that an integer can be inserted into a `double` variable without any problem (because we are going from a less specific type to a more specific one).
//Create an `int` variable named `basePing` with the value `100`.
//Immediately afterwards, create a `double` variable named `exactBasePing` and place the `basePing` variable directly into it.

int basePing = 100;

double exactBasePing= basePing;
```

```
//5
//Using the Math library:
//Use the .NET `Math` library to perform a mathematical operation on the `averageResponseTime` (for example, take its square root using `Math.Sqrt` or round it).
//Prove that this library works natively with `double` types.

double sqrAverage = Math.Sqrt(averageResponseTime);

Console.WriteLine(sqrAverage);
```

```
//6
//Comprehension question (write the answer as a `//` comment in your code):
//Suppose your system now needs to calculate the financial cost of maintaining these servers.
//Would you use the `double` type to store the amounts of money? Briefly explain why or
```

why not.

//I will not use the double variable type, I will use decimal because in everything related to money, you need to be ultra precise, and the most suitable for this level of precision and dealing with money is decimal.

//~~~~~

תרגיל 1: מלכודת הממוצע

יש לך שלושה גילאים של משתמשים במערכת.

הגילאים הם מספרים שלמים: 21, 43, ו-62.

1. כתבי קוד שמחשב את הממוצע של שלושת הגילאים האלו.

2. שמרי את התוצאה בתוך משתנה מסוג `double`.

3. **הדגש החשוב:** את חייבת לוודא שהתוצאה הסופית תכלול את הנקודה העשרונית המדויקת שלא נאבד מידע בגלל חישוב של Integers, בדיוק כמו שלמדנו.

4. הדפיסי את התוצאה לקונסולה.

תרגיל 2: כיווני המרה

כאן אנחנו מתרגלים את ההבנה של מעבר מטיפוס פחות ספציפי ליותר ספציפי.

1. צרי משתנה מסוג `int` ותני לו ערך של מספר שלם למשל 100.

2. צרי משתנה מסוג `double` וקראי לו בשם הגיוני.

3. כתבי שורת קוד שמכניסה את הערך של ה-`int` ישירות לתוך ה-`double`.

4. בהערה בקוד (`//`), כתבי משפט קצר שמסביר למה ההמרה הזו עובדת בצורה חלקה, ולמה המערכת לא מסכימה שנעשה את הפעולה ההפוכה (להכניס `double` לתוך `int`) באופן טבעי.

תרגיל 3: חדר כושר למתמטיקה

הטיפוס `double` הוא מלך המתמטיקה ב-.NET.

1. צרי משתנה מסוג `double` עם הערך `64.0`.

2. השתמשי בספריית `Math`. כדי לבצע פעולה מתמטית אחת לבחירתך למשל שורש ריבועי או כל פעולה אחרת שקופצת לך שם בהשלמה האוטומטית.

3. הדפיסי את התוצאה הסופית.

```
//Exercise 1:
//The Average Trap
//You have three ages of users in the system.
//The ages are integers: 43, 21, and 62.

intageOne = 43;

intageTwo = 21;

intageThree = 62;

//1. Write code that calculates the average of these three ages.

double averageOfThreeAges = (ageOne + ageTwo + ageThree) / 3.0;

//2. Store the result in a `double` variable.
```

```
//3. Important note: You must make sure that the final result includes the exact decimal point (so that we don't lose information due to calculating Integers, just like we learned).  
//4. Print the result to the console.  
  
Console.WriteLine(averageOfThreeAges);
```

```
//Exercise 2:  
//Conversion Directions  
//Here we practice understanding the transition from a less specific type to a more specific one.  
//1. Create a variable of type `int` and give it a value of an integer (e.g. 100).  
  
int basePingTwo = 100;  
  
//2. Create a variable of type `double` and call it a logical name.  
//3. Write a line of code that inserts the value of the `int` directly into the `double`.  
  
double newBasePingTwo = basePingTwo;  
  
//4. In a comment in the code (`//`), write a short sentence that explains why this conversion works smoothly, and why the system does not agree with us doing the opposite operation (inserting `double` into `int`) naturally.  
  
//The reason it goes smoothly is that you can go from less precise to more precise, but you can't go the other way around because the system doesn't agree to lose data.
```

```
//Exercise 3:  
//Math Gym  
//The `double` type is the king of math in .NET.  
//1. Create a `double` variable with the value `64.0`.  
  
double averageResponseTimeTwo = 64.0;  
  
//2. Use the `Math` library to perform one mathematical operation of your choice (such as square root or any other operation that pops up for you in the autocomplete).  
  
double newMath = Math.Sqrt(averageResponseTimeTwo);  
  
//3. Print the final result.  
  
Console.WriteLine(newMath);
```